

MovePro DMV Movers: Full Website and Booking System

This package contains a complete static frontend website, Node.js backend booking API, DynamoDB setup script, deployment scripts, and Jenkins pipeline for a moving business serving Maryland, DC, and Northern Virginia.

1. Project layout

```
movepro-website/  
frontend/ -> website files for S3 or CloudFront  
backend/ -> Node.js Express API for quote and booking  
scripts/ -> AWS/DynamoDB and EC2 helper scripts  
Jenkinsfile  
README.md
```

2. AWS architecture

Recommended starter architecture: customer browser loads the frontend from S3 or CloudFront. The booking form calls the backend API running on EC2. The API saves booking requests into DynamoDB.

Layer	Recommended AWS service	Purpose
Frontend	S3 + CloudFront	Host the public website
Backend	EC2 + Node.js + PM2	Run booking API
Database	DynamoDB	Store booking requests
CI/CD	Jenkins	Deploy frontend and backend
Security	IAM role + security groups	Least privilege and controlled access

3. Backend setup on EC2

Use Amazon Linux 2023. Attach an IAM role to the backend instance that can read/write the MoveProBookings DynamoDB table. Open SSH only from your IP. For early testing, open TCP 5000 only from your IP, then later move behind HTTPS.

```
sudo dnf update -y  
sudo dnf install -y git nodejs npm  
sudo npm install -g pm2
```

4. DynamoDB table

Run this from your local machine or Jenkins instance where AWS CLI is configured.

```
cd scripts
chmod +x create-dynamodb-table.sh
AWS_REGION=us-east-1 BOOKINGS_TABLE=MoveProBookings ./create-dynamodb-table.sh
```

5. Backend deployment

Copy the backend folder to /home/ec2-user/movepro-website/backend on the EC2 backend server. Create .env from .env.example and update allowed origins after the frontend domain is known.

```
cd backend
cp .env.example .env
npm install
npm start
```

Production with PM2:

```
cd /home/ec2-user/movepro-website/backend
chmod +x deploy-backend.sh
./deploy-backend.sh
pm2 status
```

6. Frontend deployment

After the backend has a public IP or domain, edit frontend/app.js and set API_BASE_URL to your backend API URL.

```
const API_BASE_URL = "http://YOUR_BACKEND_PUBLIC_IP:5000";
```

Then upload the frontend folder to S3:

```
cd frontend
chmod +x deploy-frontend.sh
./deploy-frontend.sh YOUR_S3_BUCKET_NAME
```

7. Jenkins pipeline

The included Jenkinsfile expects an SSH credential named backend-ec2-key. Replace S3_BUCKET and BACKEND_HOST inside the Jenkinsfile. Use an IAM role on Jenkins for AWS access where possible.

8. Next production upgrades

Add HTTPS using CloudFront and ACM for the frontend. Put the backend behind an Application Load Balancer or Nginx with TLS. Add Stripe deposits, admin authentication, SMS/email notifications, and CloudWatch alarms.